

METIN-LIKE MVP - Technical Spike & Tiny MMO Fork Plan

Godot 4 | multiplayer 3D | Metin 2 DNA | PoE-lite item economy

Historical planning document, translated into English. Later accepted decisions in `project-direction.md` and `persistence-policy.md` take precedence; this translation does not revive superseded target-attack, direct-loot or early-crafting proposals.

Decision: FORK Tiny MMO as an infrastructure donor rather than a ready-made gameplay base. Open-MT2 remains a reference for Metin behavior/domain (drops, inventory, quests, shops, protocol), not the target server.

[Tiny MMO repository](#) | [Open-MT2 repository](#)

1. Architectural decision

Fork Tiny MMO, but not by simply replacing a knight sprite with a Warrior. Its networking/server layer is valuable, while its gameplay is strongly 2D/action-RPG. The goal is a 3D Metin-like, so infrastructure and game domain are separated from the first spike.

```
Tiny MMO fork
|
+-- KEEP: gateway / master / world lifecycle
+-- KEEP: protocol / endpoints / wire codec
+-- KEEP: replication / AOI / instance management
+-- KEEP: auth / admin / basic persistence plumbing
|
+-- REPLACE: CharacterBody2D -> CharacterBody3D
+-- REPLACE: 2D movement/combat/AI
+-- REPLACE: inventory/item model
+-- REPLACE: quests/progression/content
+-- DELETE: demo assets and unwanted game-specific systems
```

Why fork instead of copying a few files? Initially it is unclear which dependencies are genuinely loose. A fork preserves a working baseline, history and upstream comparison. More aggressive pruning can follow a successful spike.

2. Spike goal

Determine whether Tiny MMO infrastructure really shortens the path to a 3D Metin-like, or its 2D gameplay is so coupled that a thinner custom networking layer would be cheaper.

```
Client A ----\
               > Gateway -> Master -> World
Client B ----/
```

Minimal technical vertical slice:

1. Login.
2. Shared 3D map.
3. Mutual movement replication.
4. Server-side movement validation.
5. One mob.
6. Server-side combat.
7. Death and loot roll.
8. ItemInstance in inventory.
9. Equip / upgrade / affix reroll.
10. Reconnect and persistence.

3. Spike scope, step by step

3.1. 3D multiplayer

Create an empty World3D and MetinPlayer : CharacterBody3D. Publish entity_id, position, rotation, velocity and animation_state. Clients may predict movement, but the server maintains canonical state and validates max_speed, delta distance, map bounds and teleport threshold.

3.2. First mob

One WildDog with IDLE -> AGGRO -> CHASE -> ATTACK -> RETURN. AI runs only on the world server. Separate MobDefinition (static content) from MobEntity (runtime state).

3.3. Combat

The original proposal sends AttackRequest(target_entity_id). The server checks range, cooldown and attacker/target state before calculating damage. Clients handle animation/VFX, not combat results.

3.4. Item model

Use ItemDefinition + ItemInstance from the beginning for +0...+9, unique affixes, sockets, trade and economy. Tiny MMO's simple item_id + amount is not the final equipment model.

3.5. Loot

The original proposal allowed direct inventory loot during the spike, prioritizing server rolls and durable ItemInstance creation while deferring ground rendering. Later decisions explicitly require ground loot instead.

3.6. Inventory/equip

Minimal bag + weapon slot. EquipRequest goes to the server, which checks ownership, slot/requirements, assigns the equipment item UID and recalculates final stats.

3.7. PoE-lite crafting test

The original proposal includes two test currencies: Chaos Stone rerolls all random affixes; Blessed Stone adds one affix. This would test ItemInstance flexibility for the economy early. Later milestone decisions narrow/defer crafting.

3.8. Persistence

SQLite is sufficient for the spike. Separate item_instances/inventory_slots matter more than storing equipment as one JSON blob. A public server may later use PostgreSQL.

4. Minimal domain model

ItemDefinition

- id
- type / subtype
- required_level
- base_stats
- max_upgrade
- allowed_affix_groups

ItemInstance

- uid
- definition_id
- owner_id
- upgrade_level
- affixes[]
- sockets[]
- bind/trade flags
- created_at

AffixDefinition

- id
- group
- tier
- stat
- min_value
- max_value
- weight
- item_constraints

MVP needs a few clear affix groups, not hundreds: STR/DEX/VIT/INT, attack speed, crit, monster damage, boss damage, skill damage and life steal. The goal is economy and decisions, not copying PoE complexity.

5. Tiny MMO: keep, replace, delete

The original sequencing principle was to run the fork identically to upstream, then commit a separate 3D spike, then delete old gameplay after two clients connect. Each stage retains a working reference. Later owner-approved cleanup started earlier.

Status	Area	Action	Notes
KEEP	source/common/network	Wire protocol, endpoints, sync, codec.	Main reason to fork; avoid unnecessary early changes.
KEEP	source/server/gateway	Gateway/auth/routing.	Retain.
KEEP	source/server/master	Account/world orchestration and admin.	Simplify only after spike.
KEEP	source/server/world	Lifecycle, DB plumbing, server components.	Keep skeleton; assess gameplay components separately.
KEEP	addons/tinymmo	Tiny MMO plugin/tooling.	Keep through at least spike end.
KEEP	addons/godot-sqlite	Prototype SQLite.	Retain.
REPLACE	source/common/gameplay/characters	2D hierarchy.	Add CharacterBody3D alongside it, then remove 2D.
REPLACE	source/client/local_player	2D input/movement/local player.	Do not delete before a working 3D replacement.
REPLACE	source/common/gameplay/combat	Area2D/hitboxes/action-RPG combat.	Reference rewards/authority; write combat for Metin-like play.
REPLACE	source/common/gameplay/items	Item/inventory model.	Supporting reference; target ItemDefinition + ItemInstance.
REPLACE	source/common/gameplay/maps	2D maps.	New 3D maps.
REPLACE	source/client/ui	Tiny MMO UI.	Keep spike login/debug UI as needed.
DELETE	assets/sprites	Demo 2D sprites.	Confirm login/spike does not require them.
DELETE	assets/audio	Demo audio/content.	Not needed for technical spike.
DELETE	source/common/gameplay/basing	Base/territory system.	Outside MVP.
DELETE	source/common/gameplay/crafting	Tiny crafting.	Own PoE-lite model.
DELETE	source/common/gameplay/mastery	Weapon mastery.	Outside MVP scope.
DELETE	source/common/gameplay/sparring	Sparring system.	Outside MVP scope.
DELETE	source/common/gameplay/lighting	Game-specific 2D lighting.	New 3D environment.
DELETE	source/common/gameplay/weather	Game-specific weather.	Outside MVP scope.
DELETE	source/common/gameplay/time	Day/time system.	Outside MVP scope.
DELETE	source/common/gameplay/leaderboard	Rankings.	Not needed for spike/MVP core.
DELETE	source/common/gameplay/jobs	Jobs.	Does not fit Metin MVP.

6. Tiny MMO prune-map links

- [source/common/network](#)
- [source/server/gateway](#)
- [source/server/master](#)
- [source/server/world](#)
- [addons/tinymmo](#)
- [addons/godot-sqlite](#)
- [source/common/gameplay/characters](#)
- [source/common/gameplay/combat](#)
- [source/common/gameplay/items](#)
- [source/client/local_player](#)
- [source/common/gameplay/maps](#)
- [source/client/ui](#)
- [assets/sprites](#)
- [assets/audio](#)
- [source/common/gameplay/basing](#)
- [source/common/gameplay/crafting](#)
- [source/common/gameplay/mastery](#)
- [source/common/gameplay/sparring](#)
- [source/common/gameplay/lighting](#)
- [source/common/gameplay/weather](#)
- [source/common/gameplay/time](#)
- [source/common/gameplay/leaderboard](#)
- [source/common/gameplay/jobs](#)

7. Systems potentially useful later

Do not immediately delete these directories; review dependencies/pattern value first:

- [source/common/gameplay/group](#): party/group logic as a potential party foundation.
- [source/common/gameplay/guilds](#): guild service/persistence, potentially useful after MVP.
- [source/common/gameplay/trade](#): inspect before implementing our own trade.
- [source/common/gameplay/shops](#): potential NPC shop reference.
- [source/common/gameplay/quests](#): some architecture ideas may remain.
- [source/common/gameplay/dungeon](#): instance/boss patterns for later.
- [source/common/gameplay/events](#): optional admin/live-event patterns.

Classify these as QUARANTINE: do not build MVP on them, but do not discard them before reviewing dependencies and useful patterns.

8. Proposed commit order

1. Fork upstream and tag an unchanged baseline.
2. Rename project/namespace/config; preserve MIT LICENSE.
3. Add spike/3d-network branch.
4. Add World3D + MetinPlayer3D alongside existing gameplay.
5. Achieve A sees B / B sees A and server validation.
6. Add one server-side mob/simple combat.
7. Add ItemDefinition + ItemInstance/persistence.
8. Add test loot/equip/+1/Chaos Stone.
9. Then remove 2D gameplay/demo content marked DELETE.
10. Tag spike-success after cleanup; actual MVP begins there.

9. Acceptance criteria

- ☐ Tiny networking works with Node3D/CharacterBody3D.
- ☐ Two clients reliably see each other's movement.
- ☐ Server rejects obvious speed/teleport violations.
- ☐ Mob runs and decides server-side.
- ☐ AttackRequest does not trust client damage.
- ☐ Death/respawn/despawn replicate correctly.
- ☐ Server creates unique ItemInstance.
- ☐ Inventory persists across logout/login.
- ☐ Equip changes final stats.
- ☐ Upgrade level changes stats.
- ☐ Chaos Stone changes affixes only server-side.
- ☐ Reconnect duplicates neither entities nor items.

10. Go / No-Go after the spike

GO: Tiny MMO saves networking, replication, auth and world lifecycle without forcing 2D. Build on the fork.

NO-GO: Clean 3D requires rewriting replication/instance/network core. Retain Tiny MMO patterns but build a thinner custom layer.

11. References

- [Tiny MMO repository](#)
- [Tiny MMO MIT license](#)
- [Tiny MMO networking](#)
- [Tiny MMO gameplay](#)
- [Open-MT2 repository](#)
- [Open-MT2 quests](#)
- [Open-MT2 DropManager](#)

Forking Tiny MMO is sensible, but success depends on quickly separating 2D gameplay from infrastructure. The spike should settle this before content production.